AI Contact Centre Demo

Architecture, Build Plan & Team Responsibilities

Planning document — pre-implementation

Contents

1. Purpose of This Document

2. The Architecture in Plain Words

- 2.1 Voice in (Browser → the system)
- 2.2 The session gateway (a plumbing layer, no AI)
- 2.3 Speech-to-Text — STT (audio → words)
- 2.4 The Orchestrator — the brain (this is the product)
- 2.5 Tools (how the system actually does things)
- 2.6 RAG — knowledge retrieval
- 2.7 The policy engine (the guardrails)
- 2.8 Response → Text-to-Speech → voice out
- 2.9 Human handoff & observability

3. Why the Architecture Is Shaped This Way

4. Scope, Timeline & Where the Risk Sits

- 4.1 What the scope actually costs
- 4.2 A note on “barge-in”

5. Implementation Plan (Phases 0–4)

6. Who Does What

- 6.1 Ownership at a glance
- 6.2 The load is deliberately uneven — plan to rebalance

7. Week-by-Week Plan

8. Key Risks & How We Handle Them

9. What “Done” Looks Like (Demo Checklist)

Appendix A — Recommended Technology Stack

Appendix B — Technical Reference (schemas, contracts, latency)

Appendix C — Implementation Guide (repo, build order, testing)

1. Purpose of This Document

This is a planning document, written before any code. Its job is to make sure everyone on the team understands what we are building, why the pieces fit together the way they do, who owns what, and what happens in which week. Read it once end to end before you touch the repo.

The thing we are actually building is **the AI layer of a contact centre** — the part that understands a customer, decides what to do, does it, and answers back. We are not building a complete telephony product. The value we are demonstrating is enterprise AI orchestration: understanding, memory, tool use, knowledge retrieval, workflows, and guardrails.

> **One idea to hold onto while reading**
> The whole system is a loop that runs once per turn: **the customer speaks → we turn it into text → we understand it → we decide what to do → we do it → we generate an answer → we speak it back.** Every box in the architecture is just a stage of that loop. If you ever get lost, come back to this sentence.

2. The Architecture in Plain Words

Below is each stage of the loop, described in ordinary language. The technical names are in brackets so you can match them to the diagram from the architecture document.

2.1 Voice in (Browser → the system)

The customer opens a web page and clicks "Start Conversation." The browser captures their microphone and streams the audio out continuously as they talk — it does not record first and send later. Streaming matters because we want to start understanding the customer before they have finished their sentence.

2.2 The session gateway (a plumbing layer, no AI)

This is a dumb pipe. It keeps the connection open, remembers which audio belongs to which conversation, and passes messages back and forth. There is no intelligence here on purpose. Its only jobs are holding the connection and never losing track of who is who.

2.3 Speech-to-Text — STT (audio → words)

A speech service converts audio into text *as the person speaks*. It gives us rough "partial" guesses that keep updating, then a "final" version once the customer pauses. We start reacting to partials so the conversation feels fast.

2.4 The Orchestrator — the brain (this is the product)

This is roughly 80% of the real work and the only part worth showing off. Everything else is a commodity we buy or wrap. The orchestrator **owns the truth about the conversation** and decides what happens next. Internally it does five separate jobs — keep them separate in your head, because keeping them separate in the code is what makes this design good:

- **Understanding —** an LLM reads the transcript and pulls out the intent ("this is a billing complaint") and entities ("account 12345", "charged twice").

- **State —** a single structured object that everyone reads and writes: is the customer verified, what is the intent, has a ticket been created, what is the sentiment, which workflow step are we on. This is the system's short-term truth.
- **Memory —** three layers: what is happening right now (working memory), the full transcript (conversation memory), and history from past calls (long-term memory, optional).
- **Planning —** given the state and intent, the LLM decides the next move: call a tool, look something up in the knowledge base, ask a clarifying question, or hand off to a human.
- **Response generation —** a separate LLM call turns the structured result ("engineer available tomorrow") into a natural, polite sentence.

The LLM should be able to decide for a refund if the amount is less than certain threshold. Also it needs to access the policies and take actions. Above the certain amount the LLM should route it to human.

The single most important design rule

The LLM is used for **language and planning-within-limits — never for executing business logic.** The LLM never decides whether a refund is allowed. It never invents a ticket number. It chooses words and picks which tool to call. Everything with a real-world consequence is done by deterministic code. If you remember only one thing from this document, remember this.

2.5 Tools (how the system actually does things)

Each enterprise action — *lookup_customer(), check_outage(), book_engineer(), refund_payment()* — is a function the LLM can request. At first these are fake: small endpoints that return realistic canned data. Later they would point at the real CRM and billing systems. The LLM chooses *which* tool and *with what arguments*; the tool does the actual work and returns a result.

2.6 RAG — knowledge retrieval (answering questions from documents)

RAG is used only to answer questions from documents — "what is your cancellation policy?" It finds the relevant text in a knowledge base and hands it to the LLM to answer, with citations.

The line that people get wrong

RAG answers questions. Tools perform actions. "What is the refund policy?" is RAG. "Refund my money" is a tool. Blurring these two is the most common mistake in this whole space — keep them cleanly separate.

2.7 The policy engine (the guardrails)

These are hard-coded business rules that gate an action before it runs: a refund over ₹10,000 needs manager approval; three failed diagnostics forces an engineer visit; an angry customer escalates. This layer exists so the LLM cannot be talked into breaking the rules. It is deterministic on purpose.

2.8 Response → Text-to-Speech → voice out

The generated sentence goes to a text-to-speech service, which streams the audio back to the browser as it is produced, so the customer hears the reply begin almost immediately rather than waiting for the whole paragraph.

2.9 Human handoff & observability (the supporting cast)

Handoff produces a clean summary — who the customer is, what was done, current sentiment, ticket number — so a live agent can take over mid-conversation without asking the customer to repeat themselves. **Observability** logs every intent, tool call, latency figure, and token count. This is what makes the demo legible to the audience and what makes debugging possible for us.

That is the entire system: audio → text → understand → decide → act → generate → speak, with one shared state object threading through it and deterministic guardrails wrapping the actions that matter.

3. Why the Architecture Is Shaped This Way

Five principles explain almost every decision above. If a future change respects these, it is probably fine; if it breaks one, stop and think.

1. **Separate conversation, reasoning, and execution.** Speech services only handle audio. The orchestrator owns state and flow. Tools do the real-world actions. Mixing these is where systems become unmaintainable.
2. **The LLM speaks and plans; it never executes business logic.** Consequential decisions are deterministic code, not model output.
3. **Tools act; RAG answers.** Transactions go through tools; knowledge questions go through retrieval.
4. **Policies are checked before actions run.** Guardrails gate the tool, so the model cannot invent or bypass a business rule.
5. **One shared state object is the single source of truth.** Every component reads and writes the same state, so nothing has its own private, conflicting idea of what is going on.

4. Scope, Timeline & Where the Risk Sits

4.1 What the scope actually costs

The scope we are keeping is mostly the cheap part. It is worth understanding why:

- **Three workflows are not three times the work of one.** The first workflow costs us the entire orchestrator, state model, and tool plumbing. Workflows two and three are mostly new config and new mock tools on top of that. So three workflows ≈ 1.5× one workflow.
- **RAG on a small corpus is bounded and low-risk.**
- **The policy engine is small — deterministic rules.**
- **Voice is where most of the risk lives.** Streaming speech in and out, detecting when the customer has finished talking, and tuning latency until it feels conversational are the genuinely hard, genuinely unpredictable parts — and voice is mandatory, so it cannot be traded away.

4.2 A note on “barge-in”

Barge-in means letting the customer interrupt the AI mid-sentence, the way people do on real calls. Without it, the demo feels turn-based: the AI speaks, the customer waits, then talks. That is fine for a scripted demo and most demos work this way — but it should be a conscious, communicated choice, not a surprise. It is treated here as a stretch goal.

5. Implementation Plan

The organising principle is **contracts first, then a thin end-to-end skeleton, then breadth, then voice.** Juniors get blocked and step on each other when interfaces are undefined, so we fix the interfaces first, build one complete path, and only then widen and add audio.

Phase 0 — Foundations & voice spike (Week 1)

A owns this phase. Define the four contracts everyone else codes against:

- The Conversation State schema — the shared object; the spine of the system.
- The tool interface — the input/output shape every tool must conform to.
- The workflow format — workflows described in YAML/JSON, never hard-coded.
- The frontend ↔ backend event contract.

In parallel, run a throwaway **voice spike**: microphone → WebSocket → STT → dumb passthrough → TTS → speaker, measuring round-trip latency. This proves the audio pipe and surfaces vendor/latency problems in Week 1, when we have seven weeks to react, instead of Week 6, when we have two. Do not let this slip.

Phase 1 — Text-only walking skeleton, one workflow (Weeks 2–3)

Build the thinnest complete path — typed text in → orchestrator → mocked tool → LLM response → text out — for the Technical Support workflow only. This proves the brain works before we spend effort on breadth or audio, and text lets us write automated tests that audio cannot. By the end you can type “my internet is down” and watch: authenticate → check outage → book engineer → create ticket, with a ticket number on screen.

Phase 2 — Breadth: second workflow, RAG & policies (Weeks 3–5)

With one workflow proven, add the second core workflow, stand up RAG on a small policy corpus (FAISS or pgvector locally — skip a hosted search service unless we actually need it), and make the orchestrator correctly route knowledge questions to RAG and actions to tools. Build the policy engine as a deterministic rules layer consulted before gated actions, and add the human-handoff summary.

Phase 3 — Merge voice into the working product (Weeks 5–7)

Now the brain works and the pipe is proven, connect them: streaming STT feeding the orchestrator, streaming TTS on the way out. This is where the real latency tuning happens and the phase most likely to run long — which is fine, because everything before it is already demoable.

Phase 4 — Third workflow, hardening & stretch (Weeks 7–8+)

Add the third workflow, handle the unhappy paths (mishears, tool failures, low-confidence intent → ask a clarifying question), script the demo, and build the observability dashboard. Only if there is room: barge-in and compound multi-intent utterances. Treat these last two as genuine buffer, not committed scope.

> **Firm technical recommendation: use WebSocket audio, not WebRTC**
> The architecture document lists both and it is muddled. WebRTC brings connection-negotiation machinery that buys us nothing for a single-browser demo and costs us days. Route audio browser → WebSocket → backend → speech service. This keeps one control point (state, tools, and orchestration all

live on the backend) and removes the biggest avoidable voice risk. Only reach for WebRTC if a hard requirement forces it.

6. Who Does What

6.1 Ownership at a glance

Owner	Owns
A	Orchestrator (LangGraph loop, state, intent, planning); server-side voice integration (streaming STT in / TTS out, latency tuning); policy engine; both integration seams.
B	Backend breadth: FastAPI mock tools, workflow engine (YAML loader + step tracking), RAG (corpus, retrieval, citations), structured observability logging.
C	Frontend: microphone capture, audio playback, the WebSocket audio client, and the UI (transcript, workflow-progress panel, ticket display).

7. Week-by-Week Plan

Weeks are indicative. The plan is deliberately built so that from Week 3 onward there is always something to demo.

Week	A	B	C
Week 1	Define the four contracts (state schema, tool interface, workflow format, event contract). Stand up Redis + Postgres, repo, CI. Run the voice spike jointly with B.	Start the FastAPI mock tools for Technical Support against the tool interface. Learn the STT/TTS APIs.	Voice spike with A (mic → WS → STT → TTS → speaker, measure latency). Scaffold the React app.
Weeks 2–3	Build the orchestrator loop for Technical Support: state, intent extraction, plan → act → generate.	Finish Technical Support mock tools; build the workflow engine (YAML loader + step tracking).	Build the UI against fake events: transcript view, workflow-progress panel, ticket display. Text input for now.
Weeks 3–4	Wire in the second workflow's routing; build (or review) the policy engine.	Stand up RAG: corpus, retrieval, citations. Add observability logging as you go.	UI largely done — lend spare capacity to A on RAG or mock tools. Add the RAG-citations display.
Weeks 4–5	Integrate the policy engine into the tool-call path; add the human-handoff summary.	Extend mock tools for the second workflow; finish observability logging.	Refund/second-workflow UI states; polish the workflow panel.

Week	A	B	C
Weeks 5–6	Begin the voice merge: streaming STT into the orchestrator, streaming TTS out. Latency tuning.	Support the voice merge from the backend; keep logging latency per turn.	Client-side voice: mic capture and streamed audio playback, wired to the live backend.
Weeks 6–7	Continue latency tuning; handle unhappy paths (mishears, tool failures, low-confidence intent).	Third workflow tools; begin the observability dashboard.	End-of-turn UX, "AI thinking" indicator, error states in the UI.
Weeks 7–8	Demo scripting; final latency and reliability pass; buffer.	Finish the observability dashboard; unhappy-path tests.	Demo polish; presenter-facing UI details.
Stretch	Barge-in (interrupting the AI mid-sentence); compound multi-intent utterances via planner decomposition.	Extra mock integrations; richer logging.	Barge-in playback handling; multi-intent UI.

8. Key Risks & How We Handle Them

Risk	Why it matters	How we handle it
Voice latency feels sluggish	A slow round trip breaks the illusion of a real conversation — the single most visible failure in a demo.	Set a budget on day one (end-of-speech to start-of-reply-audio under ~1.5s). Measure from the Week-1 spike onward. Stream generation and TTS; minimise LLM calls per turn.
LLM invents business logic	A model that decides refunds or invents ticket numbers is a compliance and trust failure.	Policy engine gates every consequential action; tools (not the LLM) perform them; the LLM only picks and phrases.
RAG-vs-tools confusion	Answering "refund my money" with a policy paragraph, or trying to "do" a knowledge question, looks broken.	Explicit routing in the orchestrator; test both paths separately; demonstrate the distinction on purpose in the demo.
Model availability / naming	The stack names a specific model; names and access shift fast.	Confirm the actual available tool-calling model before building; hide it behind a swappable interface so we are not vendor-locked.
Barge-in expectation gap	If the use case expects natural interruption and we scoped it out, the demo disappoints.	Flag now that barge-in is a stretch goal. If it is required, fund it by cutting the third workflow.

9. What “Done” Looks Like (Demo Checklist)

Concrete, observable outcomes. If we can do all of these on stage, the demo succeeds.

- A customer speaks; the transcript appears live and the reply is spoken back with a conversational feel.
- The workflow-progress panel updates in real time (customer identified → outage checked → engineer booked) and a ticket number appears.
- A knowledge question (“what is your cancellation policy?”) is answered from documents with a citation — visibly different from a transaction.
- A guardrail fires visibly — e.g., a large refund is held for approval — showing the LLM cannot bypass business rules.
- A handoff summary is generated that a human agent could act on without asking the customer to repeat anything.
- The observability view shows intent, tool calls, latency, and token/cost per turn — proving this is engineered, not a black box.

Appendix A — Recommended Technology Stack

Carried over from the architecture document, with our one change noted: WebSocket for audio transport rather than WebRTC (see Section 5).

Layer	Technology
Frontend	React + TypeScript
Audio capture/playback	Browser Web Audio / MediaRecorder
Streaming transport	**WebSocket** *(recommended over WebRTC for a single-browser demo)*
Backend	FastAPI
Orchestration	LangGraph
LLM	A current tool-calling-capable model — confirm availability before building; keep swappable
Speech-to-Text	Azure Speech or Deepgram
Text-to-Speech	Azure Speech
RAG / vector store	FAISS or pgvector locally; hosted search only if needed
Mock enterprise APIs	FastAPI microservices
Session state	Redis
Persistent storage	PostgreSQL
Observability	OpenTelemetry + Langfuse

Appendix B — Technical Reference

This appendix pins down the abstract pieces named in the plan: the shared state, the tool interface, the workflow format, the message contract, the orchestrator loop, and the latency budget. Treat the schemas and code as starting shapes to agree on in Week 1, not final APIs. Anything vendor- or library-specific (LangGraph node API, speech SDK calls) should be confirmed against current docs before you rely on it.

B.1 One turn, end to end

A single conversational turn runs through these stages. Everything reads and writes the one shared state object (B.2).

1. Audio arrives from the browser over the WebSocket and is forwarded to the speech service.
2. STT emits partial text (ignored for now) and a final transcript once the customer pauses.
3. Understanding: an LLM call extracts intent + entities and writes them to state.
4. Routing: if no workflow is active, one is selected from the intent.
5. Planning: an LLM call decides the next action — call a tool, retrieve knowledge, ask a clarifying question, or escalate.
6. Guard: if the action is a tool call, the policy engine checks it before anything runs and may replace it (e.g. hold for approval).
7. Act: the tool (or retriever, or handoff) runs and returns a structured result, which is applied to state.
8. Generate: an LLM call turns the structured result into a natural reply.
9. Speak: the reply is streamed to TTS and back to the browser as audio; the state update (workflow step, flags, ticket) is pushed to the UI.

B.2 Conversation State schema

One object per session, held in Redis. Every component reads and writes it; it is the single source of truth.

state.py — illustrative shape

```
{
  "session_id":   "sess_abc123",
  "created_at":   "2026-01-15T09:30:00Z",
  "channel":      "voice",

  "customer": {
    "verified":    false,
    "customer_id": null,
    "tier":        null          // "standard" | "premium"
  },

  "intent": {
    "name":       null,          // "technical_support" | "billing" | ...
    "confidence": 0.0,
    "entities":   {}             // { "account_no": "12345" }
  },

  "workflow": {
    "name":            null,     // active workflow id
    "step":            null,     // current step id
    "completed_steps": []
  },
```

```
  "flags": {
    "ticket_created":   false,
    "engineer_booked":  false,
    "escalated":        false,
    "awaiting_approval": false
  },

  "sentiment":  "neutral",       // "neutral" | "frustrated" | "angry"
  "turn_count": 0,
  "transcript": []               // [{ role, text, ts }]
}
```

B.3 Tool interface contract

Every enterprise action conforms to one shape so the orchestrator can call them uniformly and the LLM can be shown a consistent schema. Tools always return a structured result with an explicit success flag — never a bare value and never a thrown exception the LLM has to interpret.

tools/ — every tool follows this contract

```
# Uniform return type for every tool.
class ToolResult(TypedDict):
    ok:    bool
    data:  dict         # present when ok is True
    error: str | None   # present when ok is False

def check_outage(area_code: str) -> ToolResult:
    # Mock now; real CRM/network API later.
    return {"ok": True, "data": {"outage": False}, "error": None}

# What the LLM is shown for tool selection:
{
  "name": "check_outage",
  "description": "Check for a known network outage in an area.",
  "input_schema": {
    "type": "object",
    "properties": { "area_code": {"type": "string"} },
    "required": ["area_code"]
  }
}
```

B.4 Workflow definition format

Workflows are data, not code, so a new business process is a new file rather than a code change. Each step names the tool it calls and where to go next. The orchestrator walks the graph; it does not contain the business flow hard-coded.

workflows/technical_support.yaml

```
name: technical_support
description: Diagnose and resolve a connectivity problem.
entry_step: authenticate

steps:
  authenticate:
    goal: Verify the caller identity.
    tool: lookup_customer
    on_success: check_outage
    on_fail: escalate

  check_outage:
```

```
    goal: Rule out a known area outage.
    tool: check_outage
    on_success: run_diagnostics
    branch:
      outage_found: create_ticket

  run_diagnostics:
    goal: Attempt automated fixes.
    tool: run_diagnostics
    max_attempts: 3
    on_exhausted: book_engineer   # policy: 3 fails -> visit

  book_engineer:
    goal: Schedule a field visit.
    tool: book_engineer
    on_success: create_ticket

  create_ticket:
    goal: Record the interaction.
    tool: create_ticket
    on_success: confirm

  confirm:
    goal: Summarise the outcome to the customer.
    terminal: true
```

B.5 Frontend ↔ backend message contract

A single WebSocket carries typed JSON messages both ways (audio is base64 within the message, or a parallel binary channel if you prefer). Fixing these types in Week 1 is what lets the frontend build against fakes before the orchestrator exists.

Client → Server

```
{ "type": "control", "action": "start" }
{ "type": "audio_chunk", "seq": 12, "data": "<base64 pcm>" }
{ "type": "control", "action": "stop" }
```

Server → Client

```
{ "type": "transcript_partial", "text": "my internet is..." }
{ "type": "transcript_final",   "text": "my internet is down." }
{ "type": "assistant_text",     "text": "Let me check for outages." }
{ "type": "audio_chunk",        "seq": 3, "data": "<base64 pcm>" }
{ "type": "state_update",       "workflow_step": "check_outage",
                                "flags": {"ticket_created": false} }
{ "type": "ticket",             "id": "INC-10284" }
```

B.6 Orchestrator turn loop

The three LLM calls are deliberately separate: understanding, planning, and generation are different jobs with different prompts, and separating them is what keeps each one testable and cheap. The policy check sits between planning and acting so the model can never reach a tool without passing the guardrail.

orchestrator/ — the loop, in pseudocode

```
on final_transcript(text):
    state.transcript.append(role=user, text=text)

    # 1. UNDERSTAND   (LLM call #1 - small, fast model)
    state.intent = extract_intent_entities(text, state)
```

```
    # 2. ROUTE
    if state.workflow.name is None:
        state.workflow = select_workflow(state.intent)

    # 3. PLAN         (LLM call #2 - choose next action)
    action = plan_next(state)      # tool | rag | ask | escalate

    # 4. GUARD        (deterministic - runs before any tool)
    if action.kind == 'tool':
        verdict = policy.check(action, state)
        if verdict.blocked:
            action = verdict.required_action   # e.g. await_approval

    # 5. ACT
    result = dispatch(action)      # tool / retriever / handoff
    state.apply(result)

    # 6. GENERATE     (LLM call #3 - reply from structured context)
    reply = generate_reply(state, result)

    # 7. SPEAK + push state to UI
    stream_tts(reply)
    push_state_update(state)
```

B.7 RAG vs tools — the routing rule

This is a one-line decision the planner makes and it must be explicit, because getting it wrong is the most visible failure. If the customer is asking to know something, retrieve and answer with a citation. If they are asking to do something, call a tool. A question never triggers a transaction; a transaction never returns a policy paragraph.

- **Knowledge (→ RAG):** chunk the policy documents, embed them into the vector store, retrieve the top matches at query time, and pass them to the LLM to answer with the source cited.
- **Action (→ tool):** map the request to a tool call with typed arguments; the tool performs the deterministic work.

B.8 Policy engine

A small set of deterministic rules evaluated before a gated tool runs. Rules return either allow, or a replacement action. The LLM is never asked whether a rule applies — that would let it be talked out of the rule.

policies/ — illustrative rules

```
rules:
  - when: action == 'refund_payment' and amount > 10000
    then: require_manager_approval

  - when: sentiment == 'angry'
    then: escalate_to_human

  - when: diagnostics_failed >= 3
    then: force_engineer_visit
```

B.9 Latency budget

The conversational-feel target is roughly 1.5 seconds from the customer finishing speaking to the first audio of the reply. These per-stage figures are illustrative targets to measure against from the Week-1 spike onward,

not guarantees — the point is to have a budget and watch it, and to overlap stages (start generating before planning fully resolves, start TTS on the first sentence) rather than run them strictly in series.

Stage	Target	How we protect it
STT final after end-of-speech	~150 ms	Streaming STT with endpoint detection; do not wait for a full sentence.
Understand (intent + entities)	~150 ms	Small, fast model; tight prompt; classify, do not chat.
Plan (next action)	~200 ms	Rules-first where possible; small model for the rest.
Tool call	~200 ms	Local mocks now; run independent tools in parallel.
Generate (first token)	~300 ms	Stream tokens; never wait for the whole reply.
TTS (first audio)	~300 ms	Streaming TTS; synthesise the first sentence immediately.
Total to first audio	**~1.3 s**	*Overlap stages; the sum is a ceiling, not a sequence.*

Appendix C — Implementation Guide

A practical, ordered how-to that turns the phase plan (Section 5) into concrete steps, a repository layout, and a definition of done for each milestone. This is what the team follows day to day.

C.1 Repository layout

One repo, clear seams that match the ownership split.

```
contact-centre-demo/
|- docker-compose.yml        # redis, postgres
|- .env.example             # keys: LLM, STT, TTS (never commit real keys)
|- backend/
|  |- app/
|  |  |- main.py            # FastAPI + WebSocket gateway
|  |  |- orchestrator/      # the brain (LangGraph)
|  |  |- tools/             # mock enterprise APIs (ToolResult)
|  |  |- workflows/         # *.yaml workflow definitions
|  |  |- rag/               # retriever + policy corpus
|  |  |- policies/          # deterministic rules
|  |  |- state.py           # ConversationState schema
|  |  |- telemetry.py       # structured per-turn logging
|  |- tests/                # text-level orchestrator tests
|- frontend/
|  |- src/
|  |  |- AudioClient.ts     # mic capture + WebSocket
|  |  |- player.ts          # streamed audio playback
|  |  |- components/        # transcript, workflow panel, ticket
|- docs/
   |- architecture.md       # this plan, in the repo
```

C.2 Local environment

Everything runs locally for the demo. Redis holds session state; Postgres holds anything persistent (transcripts, tickets). Both come up from one compose file so a new machine is running in minutes.

bring the stack up

```
cp .env.example .env          # fill in LLM / STT / TTS keys
docker compose up -d          # redis + postgres
cd backend && uvicorn app.main:app --reload
cd frontend && npm install && npm run dev
```

C.3 Build order & definition of done

The order matters more than the dates. Each milestone has a concrete, demonstrable definition of done — do not move on until it is met, because later work assumes it.

Milestone	Do this	Definition of done
M0 Contracts	Write v0.1 of state schema, tool interface, workflow format, and the WS message contract (B.2–B.5).	All four are in the repo and the team has read them; the daily contract-drift sync is running.
M1 Voice spike	Mic → WebSocket → STT → passthrough → TTS → speaker; measure round-trip latency.	You can speak and hear it come back, with a latency number recorded.

Milestone	Do this	Definition of done
M2 Text skeleton	Orchestrator loop (B.6) for Technical Support, mocked tools, text in / text out.	Typing “internet is down” drives authenticate → outage → engineer → ticket, with a ticket number shown.
M3 Breadth	Second workflow; RAG on a small corpus; policy engine; handoff summary.	A knowledge question is answered with a citation; a gated action is held by a policy — both visibly distinct.
M4 Voice merge	Connect streaming STT and TTS to the working orchestrator.	The full spoken loop works end to end within the latency budget on the demo network.
M5 Harden	Unhappy paths, third workflow, observability dashboard, demo script.	Every item on the Section 9 demo checklist passes on a clean run-through.

C.4 The voice spike, step by step (Week 1)

This is the single most important week-1 task because it retires the biggest risk early. Do it in this order and stop to measure at the end.

1. Frontend: capture microphone audio in the browser and confirm chunks are being produced.
2. Open a WebSocket to a bare backend endpoint; stream the audio chunks; have the server log “received N bytes.”
3. Backend: forward the incoming audio to the streaming STT service; log the partial and final transcripts.
4. Send the final transcript straight to TTS (no LLM yet) and stream the audio back over the same WebSocket.
5. Frontend: play the returned audio.
6. Measure: time from end-of-speech to first returned audio. Write the number down; it is your baseline for the whole project.

C.5 Testing strategy

The reason we build text-first is that text is testable in a way audio is not. Lean on that hard.

- **Orchestrator tests run on text.** Feed transcripts in, assert on the resulting state and tool calls. This is your fastest, most valuable test suite and it exists before voice does.
- **Contract tests guard the seams.** A tiny test that a tool returns the ToolResult shape, and that server messages match the B.5 types, catches the silent-divergence failure mode.
- **Latency is measured, not assumed.** Keep the spike’s timing harness; log per-stage timings (telemetry.py) on every turn so a regression is visible immediately, not at the demo.
- **Policy and routing get explicit cases.** One test that a large refund is held, one that a knowledge question routes to RAG and an action routes to a tool — these are the behaviours the demo hinges on.